

STRUCTURA SI ORGANIZAREA CALCULATOARELOR

Calculatoare 3 2025-2026

CURS 2 Sapt 2 7 octombrie 2025 14:00-16:00

serban@upit.ro

Modul de notare:

Examen:	50%	
Proiect	20%	A3
Laborator:	20%	A2
Test (Midterm):	10%	A1

Obs. Fiecare componenta a notei finale (A1, A2, A3 si Examen) trebuie sa fie promovata cu nota minima 5.

Planificarea activitatilor:

Laborator:

- saptamanal, conform orarului;
- sapt. 13, 14 refaceri laboratoare;

Proiect:

- conform orarului

Curs:

- saptamanal, conform orarului;

Test (Midterm): in saptamana a 11-a (9 decembrie - 13 decembrie)

Continut

Bazele aritmetice ale calculatoarelor electronice

- Sisteme de numeratie

- Reprezentarea numerelor in virgula fixa / mobila

- Operatii aritmetice in virgula fixa / mobila

Proiectarea blocurilor logice pentru operatiile aritmetice in calculatoarele electronice

- Adunarea; scaderea

- Inmultirea

- Impartirea

Proiectarea procesoarelor

- Procesoare RISC

- Setul de instructiuni

- Calea de date (Data Path)

- Calea de control (Control Path)

Bibliografie

David A. Patterson, John L. Hennessy - Computer Organization and Design
The Hardware Software Interface [RISC-V Edition] (6th edition Morgan
Kaufmann, 2018)

David Harris, Sarah Harris - Digital Design and Computer Architecture
(2007)

J. Hennessy, D. Patterson - Computer Architecture. A Quantitative Approach
(6th edition Morgan Kaufmann 2019)

William Stallings - Computer Organization and Architecture Designing for
Performance (10th edition Pearson 2016)

A. Tanenbaum, T. Austin - Structured Computer Organization, (6th edition
Pearson 2013)

Karbo M.B. - A complete illustrated guide to the PC hardware (2005)

BAZELE ARITMETICE ALE CALCULATOARELOR ELECTRONICE

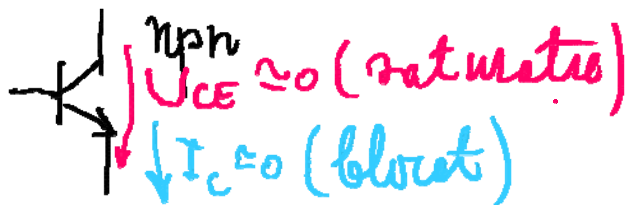
Sisteme de numeratie

Sistem de numeratie: totalitatea regulilor de reprezentare a numerelor cu ajutorul unor simboluri numite cifre (digiți).

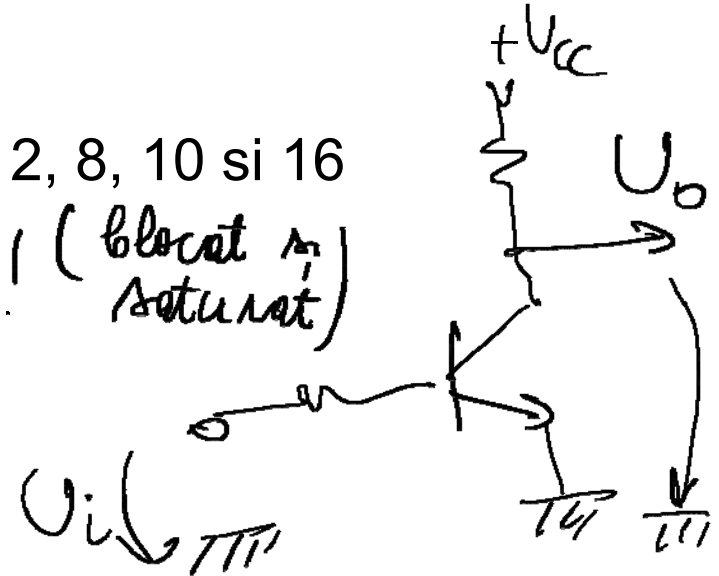
Cifra (digit): simboluri de tip întregi prin care se reprezintă numerele, pozițiile în număr indicând ordinele de mărime corespunzătoare.

Baza sistemului de numeratie: numărul de simboluri ce permit pentru reprezentarea cifrelor.

În sisteme digitale se utilizează bazele 2, 8, 10 și 16



$$P_d = U_{CE} \cdot I_C \approx 0 \quad (\text{blocat și saturat})$$



b=10	b=2	b=8	b=16
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12
19	10011	23	13
20	10100	24	14
21	10101	25	15
...	...	...	...

$$\underbrace{1011}_{8} \underbrace{0101}_{5}_{(2)} = 85_{10}$$

$$\underbrace{1011}_{26} \underbrace{0101}_{5}_{(2)} = 265_{10}$$

Schimbarea bazei de numeratie

Separat pentru partea intreaga si separat pentru partea subunitara!

N numar intreg fara semn in baza $x \rightarrow$ noua baza y .

NUMERE INTREGI

Reprezentarea N (zecimal) in baza y: $a_n a_{n-1} \dots a_1 a_0$ $a_n, a_{n-1}, \dots, a_1, a_0$ -
digiti

$$N = a_n y^n + a_{n-1} y^{n-1} + \dots + a_1 y + a_0$$

=> impartiri succesive la noua baza y, retinand la fiecare operatie restul!

$$N / y = a_n y^{n-1} + a_{n-1} y^{n-2} + \dots + a_1 + a_0 / y = N_1 + a_0 / y \quad (a_0 / y - \text{rest!})$$

=> a_0 (cifra cea mai putin semnificativa a rezultatului)

$$N_1 / y = a_n y^{n-2} + a_{n-1} y^{n-3} + \dots + a_2 + a_1 / y = N_2 + a_1 / y \quad (a_1 / y - \text{rest!})$$

.....

$$N_k / y = a_n y^{n-k-1} + a_{n-1} y^{n-k-2} + \dots + a_{k+1} + a_k / y$$

Conversia se incheie cand se obtine catul 0.

Exemplu. $N = 41_{(10)} \rightarrow$ noua baza 2.

41	:	2	=>	cat = 20	si rest = 1	=>	$a_0 = 1$	bucura (LSB)
20	:	2	=>	cat = 10	si rest = 0	=>	$a_1 = 0$	
10	:	2	=>	cat = 5	si rest = 0	=>	$a_2 = 0$	
5	:	2	=>	cat = 2	si rest = 1	=>	$a_3 = 1$	bucura (MSB)
2	:	2	=>	cat = 1	si rest = 0	=>	$a_4 = 0$	
1	:	2	=>	cat = <u>0</u>	si rest = 1	=>	$a_5 = 1$	

=> $N = 101001_{(2)}$. Verificare:

$$N = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 41$$

corect!

$$N = a_5 a_4 a_3 a_2 a_1 a_0$$

$$N = 101001_{(2)} = 41_{(10)}$$

$41_{(10)} \rightarrow$ baza 16

$41 : 16 \Rightarrow \text{cat} = 2 \text{ rest } 9 \uparrow$
 $2 : 16 \Rightarrow \text{cat} = 0 \text{ rest } 2 \uparrow$

$41_{(10)} = 29_{(16)} = 10.1001_{(2)}$

$$41 = 41_{(10)} = ?_{(8)}$$

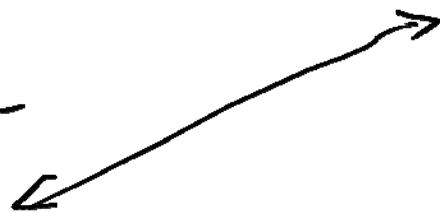
$$41 : 8 = 5 \text{ rest } 1$$

$$5 : 8 = 0 \text{ rest } 5$$

$$N = 41_{(10)} = 51_{(8)}$$

$$N = \underbrace{101}_{5} \underbrace{001}_{1}_{(2)}$$

$$\underline{\underline{51}}_{(8)}$$



$$N = 41_{(10)} = ?_{(16)}$$

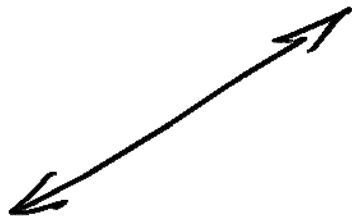
$$41 : 16 = 2 \text{ rest } 9$$

$$2 : 16 = 0 \text{ rest } 2$$

$$N = 41_{(10)} = 29_{(16)}$$

$$N = \underbrace{101}_{2} \underbrace{001}_{9}_{(2)}$$

$$\underline{\underline{29}}_{(16)}$$



Aplicatie. algoritm in pseudocod pentru conversia unui numar intreg din zecimal in binar. **TEMA 1** – Implementare in C !

pentru $n > |2^{24}|$

```
citeste nr
i = 0
cat timp nr  $\geq 2^i$  executa
|
|   a[i] = 0
|   i = i + 1
i = i - 1; n = i; a[n] = 1; nr = nr - 2i
cat timp nr  $\neq 0$  executa
|
|   i = i - 1
|   daca nr  $\geq 2^i$  atunci
|   |
|   |   a[i] = 1
|   |   nr = nr - 2i
scrie (a[i], i = n, 0)
```

NUMERE SUBUNITARE

N numar subunitar, fara semn in baza $x \rightarrow$ noua baza y .

Reprezentarea N in baza y : $0. a_{-1}a_{-2}\dots a_{-m}$

$$N = a_{-1}y^{-1} + a_{-2}y^{-2} + \dots + a_{-m}y^{-m}$$

=> inmultiri succesive cu noua baza y , retinand de fiecare data partea intreaga a rezultatului!

$$N \cdot y = a_{-1} + a_{-2}y^{-1} + a_{-3}y^{-2} + \dots + a_{-m}y^{-m+1} = a_{-1} + N_1$$

=> a_{-1} (cea mai semnificativa)

$$N_1 \cdot y = a_{-2} + a_{-3}y^{-1} + a_{-4}y^{-2} + \dots + a_{-m}y^{-m+2} = a_{-2} + N_2$$

.....

$$N_k \cdot y = a_{-k-1} + a_{-k-2}y^{-1} + a_{-k-3}y^{-2} + \dots + a_{-m}y^{-m+k+1}$$

Exemplu. $N = 0.37_{(10)} \rightarrow$ baza 2 (rezultatul pe 7 biti).

$0.37 \times 2 = 0.74$	$\Rightarrow a_{-1} = 0$
$0.74 \times 2 = 1.48$	$\Rightarrow a_{-2} = 1$
$0.48 \times 2 = 0.96$	$\Rightarrow a_{-3} = 0$
$0.96 \times 2 = 1.92$	$\Rightarrow a_{-4} = 1$
$0.92 \times 2 = 1.84$	$\Rightarrow a_{-5} = 1$
$0.84 \times 2 = 1.68$	$\Rightarrow a_{-6} = 1$
$0.68 \times 2 = 1.36$	$\Rightarrow a_{-7} = 1$

Calculul se finalizează când rezultatul este de forma $n.00$!
Iar, de multe ori nu se ajunge la această forma.
(1.00)

$\Rightarrow N \approx .0101111_{(2)}$. Verificare:

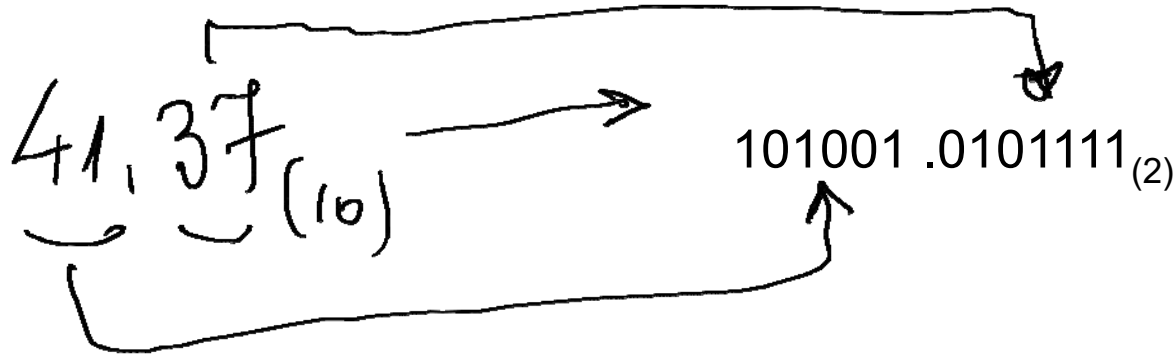
$$\begin{aligned} N &\approx 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 0 \cdot 2^{-3} + 1 \cdot 2^{-4} + 1 \cdot 2^{-5} + 1 \cdot 2^{-6} + 1 \cdot 2^{-7} = \\ &= (0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) / 2^7 = \\ &= 47 / 128 \approx 0.367 \end{aligned}$$

Conversia se incheie:

- partea subunitara a rezultatului inmultirii egala cu zero ;
- s-a calculat numarul propus de cifre (s-a atins precizia dorita).

Obs.

conversia numerelor intregi: se face exact;
conversia numerelor subunitare: aproximativ.



Aplicatie. algoritm pentru conversia unui numar subunitar din zecimal in binar, cu precizia pe m biti. **TEMA 2 – Implementare in C !**

```
citeste nr, m
pentru i=1,m executa
|   a[i] = 0
i = 0
cat timp (nr  $\neq$  0) si (i < m) executa
|   i = i + 1
|   daca nr  $\geq$  2-i atunci
|   |   a[i] = 1
|   |   nr = nr - 2-i
scrie (a[i], i = 1,m)
```

Exemplu. $N = 41.37_{(10)} \rightarrow$ baza 2 (partea subunitara pe 7 biti).

$\Rightarrow N \approx 101001.0101111_{(2)}$.

Tema 3. Conversia in binar a numarului subunitar $0.64_{(10)}$ pana cand conversia nu se poate realiza; ce observati?

Cand se finalizeaza conversia unui numar subunitar, fara aproximare (exact)? De ex. $0.375_{(10)}$

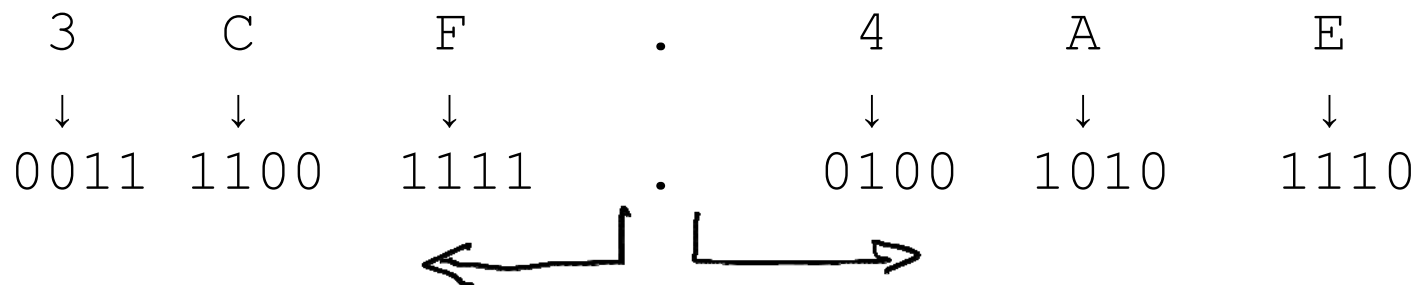
In ce conditii apar cele 2 situatii expuse mai sus?

Cazuri particulare de schimbare a bazei de numeratie

1) $x = y^n \Rightarrow$ fiecare cifra in baza $x \sim$ grup de n cifre in baza y .

Exemplu. $N = 3CF.4AE_{(16)} \rightarrow$ baza 2 $(x=16; y=2; n=4)$

$$16 = 2^4 \Rightarrow n = 4$$



$$\Rightarrow N = 1_{(2)}.$$

111001111.01001010111

Conversia s-a facut exact !

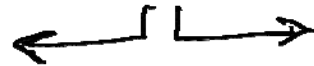
$$2) x^n = y. \quad (x=2; y=8; n=3)$$

-in vechea baza $x \Rightarrow$ grupuri de cate n cifre de la punctul zecimal spre stanga pentru partea intreaga si de la punctul zecimal spre dreapta pentru partea subunitara (daca este necesar se vor completa grupurile extreme cu zerouri);

-fiecare grup $\rightarrow$ cifra in noua baza y .

Rezultatul este exact!

Exemplu. $N = 11001111.1101011_{(2)} \rightarrow$ in baza 8.



$$2^3 = 8 \Rightarrow n = 3.$$

0 11	001	111	.	110	101	1 00
↓	↓	↓		↓	↓	↓
3	1	7	.	6	5	4

$$\Rightarrow N = 317.654_{(8)}$$

$N = 11001111.1101011_{(2)} \rightarrow$ in baza 16.

$$2^4 = 16 \Rightarrow n = 4$$

11001111₍₂₎

.0101111₍₂₎

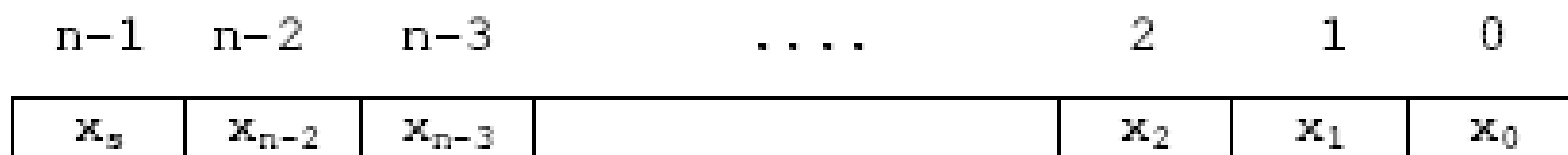
Reprezentarea numerelor zecimale

2 cazuri posibile

- In virgula fixa
- In virgula mobila

Reprezentarea numerelor in virgula fixa

Numerele: *in virgula fixa* (numere intregi sau numere subunitare) si *in virgula mobila* (numere reale).



↑
pozitia punctului zecimal
pentru un numar subunitar

↑
pozitia punctului zecimal
pentru un numar intreg

Valoarea in zecimal:

$$|x| = \begin{cases} \sum_{k=0}^{n-2} x_k 2^k & \text{pentru numar intreg} \\ \sum_{k=0}^{n-2} x_k 2^{k-n+1} & \text{pentru numar subunitar} \end{cases}$$

Reprezentarea in cod direct (cu semn si modul – marime - SM)

Codul direct permite reprezentarea explicita a numerelor prin semn si modul.

$$[x]_d = \begin{cases} 0x_{n-2}x_{n-3}\dots x_1x_0 & \text{daca } x \geq 0 \\ 1x_{n-2}x_{n-3}\dots x_1x_0 & \text{daca } x < 0 \end{cases}$$

Exemple. Sa se reprezinte in cod direct numerele $x = 21$ si $y = -20$ pe 6 biti.

$$[x]_d = 010101$$

$$0 \Rightarrow \text{semn } +$$

$$21_{(10)} = 10101_{(2)}$$

$$[y]_d = 110100$$

$$1 \Rightarrow \text{semn } -$$

$$20_{(10)} = 10100_{(2)}$$

Reprezentarea in cod complement fata de 1 sau **CC1** (sau cod invers)

$$[x]_i = \begin{cases} 0x_{n-2}x_{n-3}\dots x_1x_0 & \text{daca } x > 0 \\ \left. \begin{array}{l} 0000 \dots 000 \\ \text{sau} \\ 1111 \dots 111 \end{array} \right\} & \text{daca } \underline{x = 0} \\ 1\bar{x}_{n-2}\bar{x}_{n-3}\dots\bar{x}_1\bar{x}_0 & \text{daca } x < 0 \end{cases}$$

unde

Se poate stabili o relatie importanta *pentru numerele strict negative reprezentate in **CC1***. Astfel, daca $x < 0$, atunci:

$$\bar{x}_k = 1 - x_k$$

$$|x| + [x]_i = 0x_{n-2}x_{n-3}\dots x_1x_0 + 1\bar{x}_{n-2}\bar{x}_{n-3}\dots\bar{x}_1\bar{x}_0 = 1111\dots 111 = 2^n - 1$$

Exemple. Sa se reprezinte in cod invers numerele $x = 21$ si $y = -20$ pe 6 biti.

$$[x]_d = [x]_i = 010101$$

Reprezentarea numerelor pozitive in CC1 (cod invers) este identica cu reprezentarea in cod direct.

Numerele negative se reprezinta insa diferit. Pentru y se scrie mai intai reprezentarea in cod direct, apoi se inverseaza (se complementeaza fata de 1) fiecare bit al modulului.

$$[y]_d = 110100$$

$$[y]_i = 101011$$

$$[y]_d = 110100$$

Reprezentarea in cod complement fata de 2 – CC2 (cod complementar)

$$[x]_c = \begin{cases} 0x_{n-2}x_{n-3}\dots x_1x_0 & \text{daca } x \geq 0 \\ 1\tilde{x}_{n-2}\tilde{x}_{n-3}\dots\tilde{x}_1\tilde{x}_0 & \text{daca } x < 0 \end{cases}$$

unde, prin definitie,

$$1\tilde{x}_{n-2}\tilde{x}_{n-3}\dots\tilde{x}_1\tilde{x}_0 = 2^n - |x|$$

Se poate stabili o relatie importanta pentru reprezentarea numerelor negative in cod complementar. Daca $x < 0$. atunci:

def
↓

$$[x]_c = 1\tilde{x}_{n-2}\tilde{x}_{n-3}\dots\tilde{x}_1\tilde{x}_0 = 2^n - |x| = \underbrace{|x| + [x]_i + 1}_{= 2^n - 1} - |x| = [x]_i + 1$$

$|x| + [x]_i = 2^n - 1 \quad (CC_1)$

$2^n = |x| + [x]_i + 1$

$[x]_c = [x]_i + 1$

Exemple. Sa se reprezinte in CC2 numerele $x = 21$ si $y = -20$ pe 6 biti.

$$[x]_d = [x]_i = [x]_c = 010101 \quad (\text{numere pozitive}) \quad x - y = x + \underbrace{(-y)}_{\text{CC2}}$$

Din nou, reprezentarea numerelor pozitive in CC2 (cod complementar) este identica cu reprezentarea in cod direct.

Pentru a obtine reprezentarea unui numar negativ in CC2, se scrie mai intai numarul in cod direct, apoi se inverseaza bitii modulului pentru a obtine codul invers si in final se aduna o unitate, in rangul cel mai putin semnificativ (c.m.p.s.), conform relatiei stabilite mai sus.

$$\begin{array}{l} [Y]_d = 110100 \\ [Y]_i = 101011 + \\ \quad \quad \quad 1 \\ \hline [Y]_c = 101100 \end{array} \quad \begin{array}{l} (-20) !! \\ \downarrow \\ \underbrace{10001100}_{8 \text{ biti}} \end{array} \quad \begin{array}{l} \downarrow \\ [Y]_d = 110100 \\ \leftarrow 00 \\ \hline \underbrace{101100}_{6 \text{ biti}} \end{array}$$

Regula practica pentru scrierea in CC2 (numai pentru numerele negative !): se *parcurge reprezentarea numarului in cod direct de la dreapta spre stanga*, se *lasa toate zerourile neschimbate pana la prima unitate*, care de asemenea *ramane neschimbata*, iar apoi se *inverseaza toti ceilalti biti ai modulului*.